

Robótica e Automação - Trabalho Nº2

Leonardo Soares da Costa Tanaka - DRE: 121067652

Engenharia de Controle e Automação/UFRJ

Rio de Janeiro, Brasil

Dezembro de 2024

1 Introdução

Para a resolução do trabalho, foi o manipulador Kinova Gen3 Ultra lightweight (www.kinovarobotics.com) da Kinova Inc..

A estrutura cinemática deste manipulador de 7 juntas pode ser vista na figura:

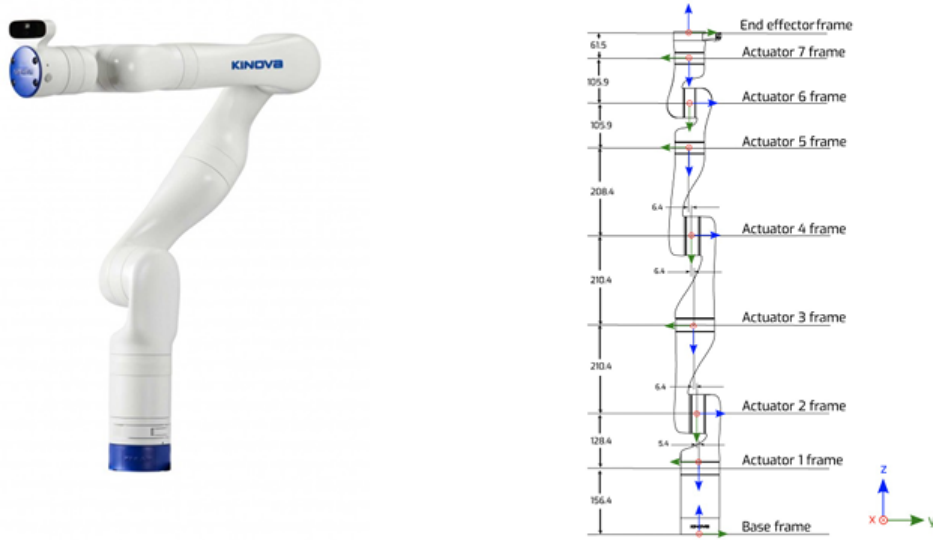


Figura 1: Foto e Diagrama do manipulador Kinova

Além disso, os parâmetros de Denavit-Hartenberg do manipulador são apresentados na Tabela 1. Foi considerado que a orientação do sistema de coordenadas 0 com relação à base é dado por $R_{b0} = R_x(\pi)$ onde $R_x(\cdot)$ é uma rotação elementar ao redor do eixo x.

Para a primeira parte do trabalho, foi considerado a posição inicial do manipulador é definida pelos ângulo das juntas: $\theta = [0 \ 15 \ 180 \ -130 \ 0 \ 55 \ 90]$ (graus). Porém, para a segunda parte do trabalho, foi considerado a posição inicial do manipulador é definida pelos ângulos das juntas: $\theta_h = [90 \ 15 \ 180 \ -130 \ 0 \ 55 \ 0]$ (graus). Além disso, foi considerado que na tabela de Denavit-Hartenberg da primeira parte do trabalho tem o $d_7 = -0.3574m$.

Junta	α (rad)	A (m)	θ (rad)	D (m)	Offset (rad)
1	$\pi/2$	0	θ_1	-0.2848	π
2	$\pi/2$	0	θ_2	-0.0118	π
3	$\pi/2$	0	θ_3	-0.4208	π
4	$\pi/2$	0	θ_4	-0.0128	π
5	$\pi/2$	0	θ_5	-0.3143	π
6	$\pi/2$	0	θ_6	0	π
7	π	0	θ_7	0	π

Tabela 1: Parâmetros de Denavit-Hartenberg Standard do Kinova Gen3

2 Desenvolvimento

2.1 Controle Cinemático de Manipuladores

Foi considerado o controle cinemático da posição do punho do manipulador em que somente a Sinais de controle é controlada, porque as 3 últimas juntas estão na mesma posição fazendo que as três últimas colunas do jacobiano sejam zero. Além disso, a velocidade máxima das juntas foi definida como $\dot{\theta}_{max} = 1 \text{ rad/s}$. Foram consideradas as seguintes trajetórias (com $\omega_n = \pi/2$):

(a) $\mathbf{x}_d$ descreve uma circunferência de 0.1 m de raio no plano $X - Z$, centrada no ponto $\mathbf{x}_0 = [0.4 \ 0 \ 0.4]^T$, e percorrida com uma frequência ω_n .

(b) A circunferência do item anterior projetada no plano definido pela equação: $(x - 0.4) - y + (z - 0.4) = 0$.

(c) Considerando agora $\omega_n = \frac{\pi}{8}$, temos: $\mathbf{x}_d(t) = \begin{bmatrix} 0.08(\sin(\omega_n t) + \sin(4\omega_n t)) + 0.4 \\ 0 \\ 0.08(\cos(\omega_n t) + \cos(4\omega_n t)) + 0.4 \end{bmatrix}$.

Foi simulado o comportamento do sistema em malha fechada para as trajetórias especificadas utilizando os seguinte controladores:

1. $\mathbf{u} = (J(\boldsymbol{\theta}))^\dagger \mathbf{K}(\mathbf{x}_d - \mathbf{x})$.
2. $\mathbf{u} = (J(\boldsymbol{\theta}))^\dagger [\dot{\mathbf{x}}_d + \mathbf{K}(\mathbf{x}_d - \mathbf{x})]$, onde $J^\dagger = J^\top (JJ^\top)^{-1}$.

Para implementar o segundo controlador, foi feita a derivação algébrica das três trajetórias consideradas no problema. Portanto, não foi utilizado derivação numérica para definir o $\dot{x}_d$ no segundo controlador. Além disso, foi sintonizado o ganho $\mathbf{K}$ com 6 configurações diferentes para obter o melhor desempenho possível atendendo a especificação de $|u_i| \leq 1 (i = 1, 2, 3, 4)$. A seguir temos os gráficos utilizados para analisar as 6 configurações diferentes:

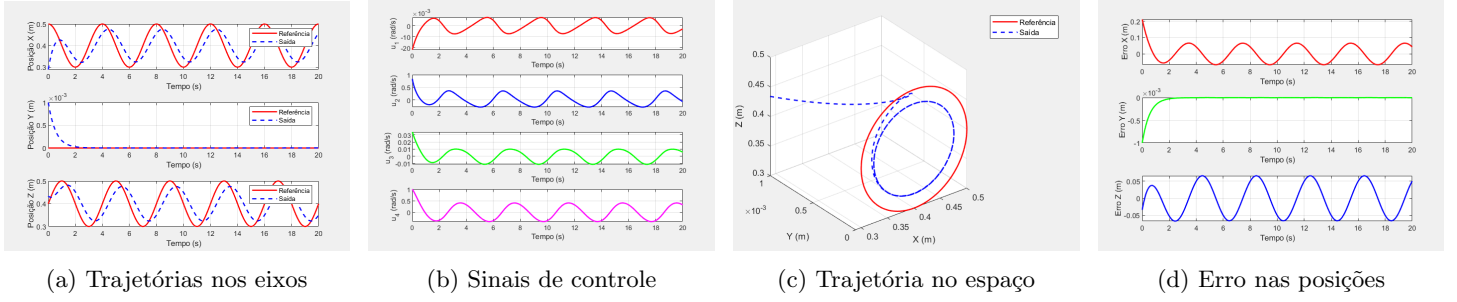


Figura 2: Gráficos da Trajetória (a) com Controlador 1 ($K = 1.81$)

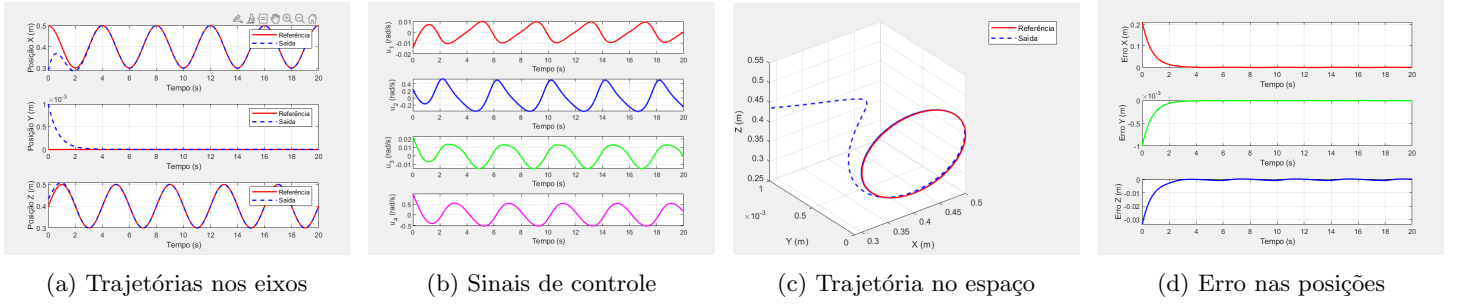


Figura 3: Gráficos da Trajetória (a) com Controlador 2 ($K = 1.39$)

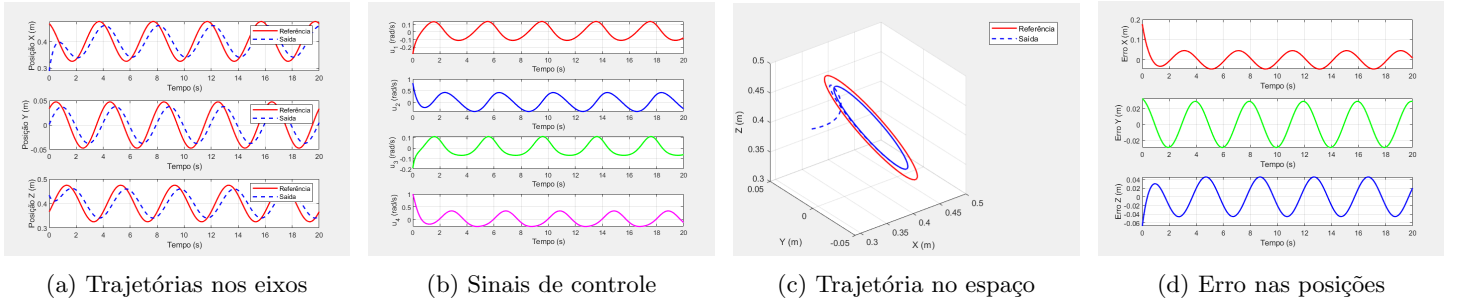


Figura 4: Gráficos da Trajetória (b) com Controlador 1 ($K = 2.05$)

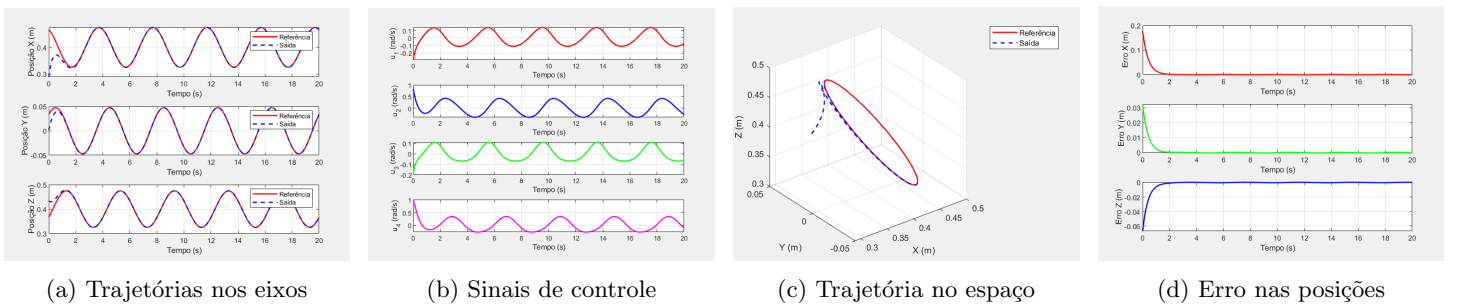


Figura 5: Gráficos da Trajetória (b) com Controlador 2 ($K = 2.42$)

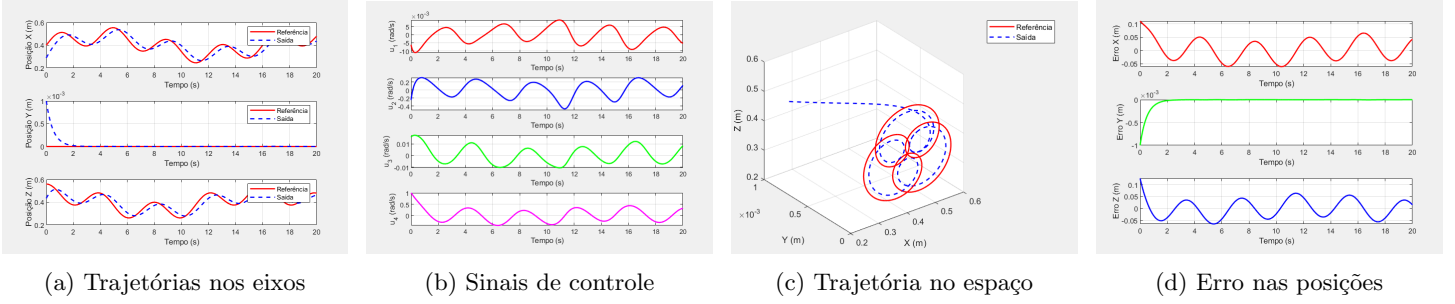


Figura 6: Gráficos da Trajetória (c) com Controlador 1 ($K = 1.99$)

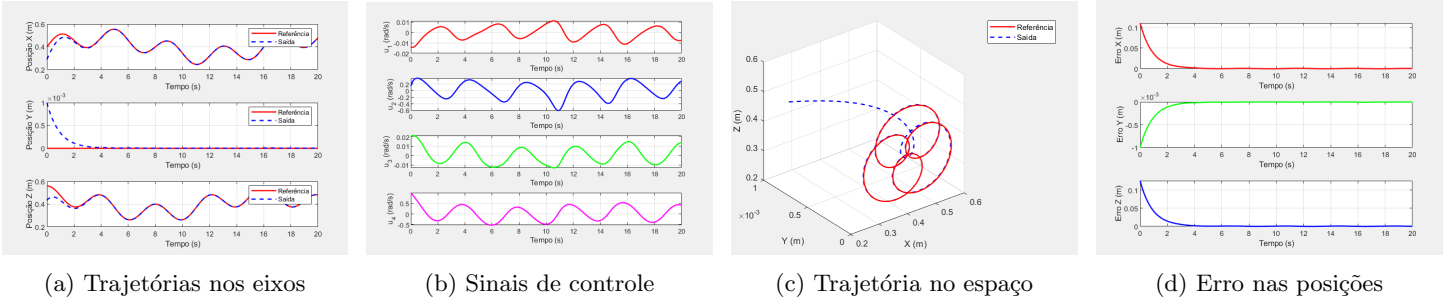


Figura 7: Gráficos da Trajetória (c) com Controlador 2 ($K = 2.17$)

Após analisar os dados obtidos das 6 configurações diferentes, é possível concluir sobre a estabilidade e desempenho de cada um dos controladores. Portanto, os controladores podem assegurar a estabilidade, se $J(\theta)$ tenha posto completo, garantindo a existência do pseudo-inverso ($J^\dagger$). Além disso, o controlador 1 não consegue ter a convergência ao ser respeitado as especificações do problema. Assim, a saída acaba tendo uma diferença de fase em relação a trajetória de referência e um desvio considerável para as três trajetórias especificadas, o que demonstra um desempenho ruim para seguimento de trajetórias. Entretanto, o controlador 2 consegue ter a convergência ao ser respeitado as especificações do problema. Ademais, o erro entre a saída e a trajetória de referência acaba tendendo a zero de uma forma bem rápida, tendo um desempenho bom para seguimento de trajetórias. Isso acontece, porque o controlador 2 possui uma feedforward ($\dot{x}_d$) compensando o movimento desejado.

2.2 Controle Cinemático de Manipuladores por Servo Visão

Para o controle cinemático da posição do efetuador utilizando servo-visão, foi considerado somente os comandos de velocidade das juntas 1,2,4,6. Assim, considerando as juntas 3,5,7 fixas, i.e. $\dot{\theta}_3 = \dot{\theta}_5 = \dot{\theta}_7 = 0$. Além disso, foi utilizada uma câmera monocular com eixo ótico (x) é perpendicular ao plano de trabalho (x-y) do manipulador, a distância focal é $f = 4mm$, posição a uma distância de $z_0 = 0.4338m$ do plano de trabalho do manipulador e o fator de escala é $\alpha = 200000$ pixels/m nas duas direções. Ademais, foi definido a origem do plano da imagem $O_c = \begin{bmatrix} 320 & 240 \end{bmatrix}^T$ e a posição da câmera com relação à base do manipulador é $P_{bc} = \begin{bmatrix} 0 & -0.603 & 0 \end{bmatrix}^T$ com orientação $\phi = 0$, i.e. $R_{bc} = I$. Foi considerada seguinte trajetória em coordenada da tela (com $\omega_n = 0.5$):

$$x_{cd} = \begin{bmatrix} 360 - 30 \cos(\omega_n t) - 30 \cos(1.5\omega_n t) - 10 \cos(2\omega_n t) \\ 150 - 20 \cos(\omega_n t + 1.6) - 20 \cos(1.5\omega_n t + 1.6) - 10 \cos(2\omega_n t + 1.6) \end{bmatrix} \quad (1)$$

Também, foi feito a derivação algébrica da trajetória para utilizar o controle 2 da subseção anterior:

$$\dot{x}_{cd} = \begin{bmatrix} 30 \sin(\omega_n t)\omega_n + 45 \sin(1.5\omega_n t)1.5\omega_n + 20 \sin(2\omega_n t)2\omega_n \\ 20 \sin(\omega_n t + 1.6)\omega_n + 30 \sin(1.5\omega_n t + 1.6)1.5\omega_n + 20 \sin(2\omega_n t + 1.6)2\omega_n \end{bmatrix} \quad (2)$$

A partir das informações acima foi feita um estudo em relação ao controle por servo-visão. Assim, temos a seguir certos conceitos que foram necessários para fazer o código de solução do problema no Matlab. A transformação câmera/espaco de trabalho é dada por:

$$\mathbf{x}_c = K_p \cdot \mathbf{x} + \mathbf{x}_{c0} \quad (3)$$

onde

$$K_p = \frac{f}{z_0} \begin{bmatrix} \alpha_1 & 0 \\ 0 & \alpha_2 \end{bmatrix} \quad (4)$$

e

$$\mathbf{x}_{c0} = \frac{f}{z_0} \begin{bmatrix} \alpha_1 & 0 \\ 0 & \alpha_2 \end{bmatrix} \begin{bmatrix} \cos(\phi) & \sin(\phi) \\ \sin(\phi) & -\cos(\phi) \end{bmatrix} P_{bc} + \begin{bmatrix} O_1 \\ O_2 \end{bmatrix} \quad (5)$$

em que

- $\mathbf{x}_c$: posição do efetuador no sistema de coordenadas da imagem.
- $\mathbf{x}_c^0$: offset da posição do frame da câmera com respeito ao do robô.
- K_p : orientação e escalamento da câmera com respeito ao robô.
- ϕ : ângulo de orientação da câmera com respeito ao frame do robô.
- f : distância focal da câmera.
- z_0 : profundidade do plano da imagem ao plano de trabalho do robô.
- $\alpha_i > 0$: fatores de escala (pixels/m).

No sistema de coordenadas da câmera, a cinemática diferencial é dada por:

$$\dot{\mathbf{x}}_c = K_p \dot{\mathbf{x}}, \quad \mathbf{x}^T = [x_1, x_2] \quad (6)$$

em que $\mathbf{x}_{cd} \in \mathbb{R}^2$ é a trajetória desejada do efetuador no plano da câmera. Uma lei de controle cartesiano pode ser projetada para que $\mathbf{x}_c \rightarrow \mathbf{x}_{cd}$, ou seja, $e_{cd} \rightarrow 0$. A equação dinâmica de e_{cd} é dada por:

$$\dot{e}_{cd} = \dot{x}_{cd} - \dot{\mathbf{x}}_c = \dot{x}_{cd} - K_p J(\theta) u \quad (7)$$

Com uma lei de controle dada por:

$$u = (K_p J(\theta))^{-1} [\dot{x}_{cd} + K(\mathbf{x}_{cd} - \mathbf{x}_c)] \quad (8)$$

O sistema em malha fechada é dado por:

$$\dot{e}_{cd} + K e_{cd} = 0 \quad \text{para } K > 0 \quad (9)$$

O sistema é exponencialmente estável, com a solução:

$$e_{cd}(t) = e^{-Kt} e_{cd}(0) \quad (10)$$

Primeiramente, foi simulado o controle por servo-visão do efetuador com $K = 1$ para avaliar a estabilidade, tendo os seguintes resultados:

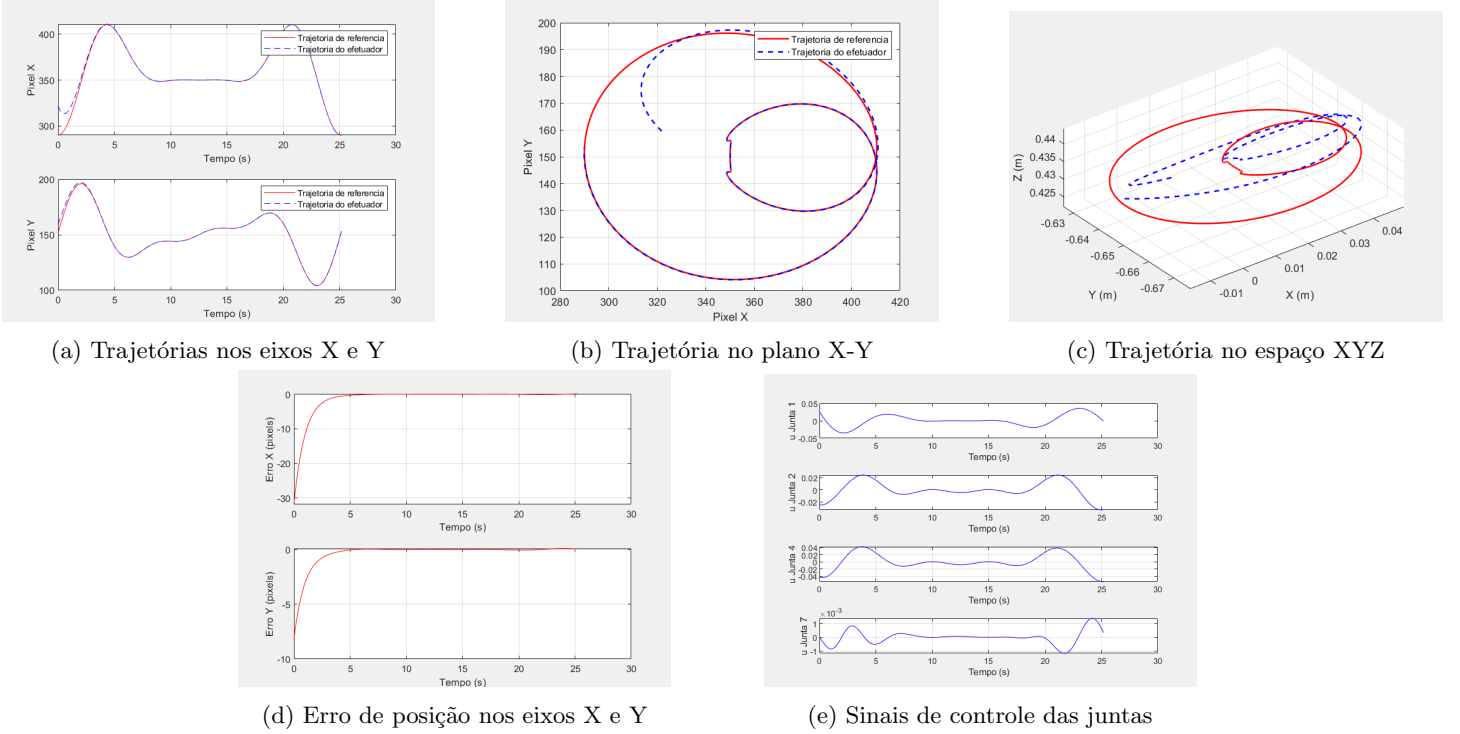
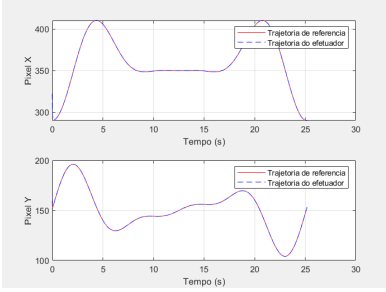
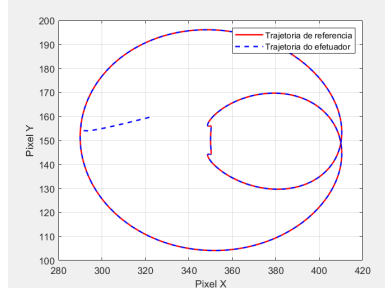


Figura 8: Gráficos do controle por servo-visão ($K = 1$)

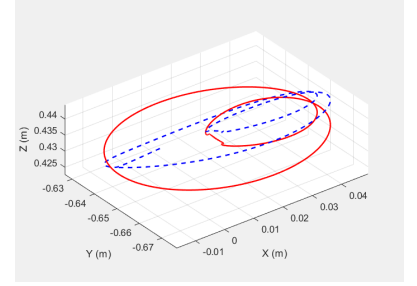
É possível notar que o efetuador consegue seguir a trajetória de referência. Além disso, o gráfico de erro consegue mostrar que o sistema é exponencialmente estável. Entretanto, para o ganho do controlador sendo $K = 1$, ainda tem um grande margem de aumento desse ganho para ficar mais próximos dos limites de $|u_i| \leq 1 (i = 1, 2, 4, 7)$. Então, foi feito a sintonização do ganho do controlador satisfazendo a expressão anterior, obtendo $K = 37.25$ e tendo os seguintes gráficos:



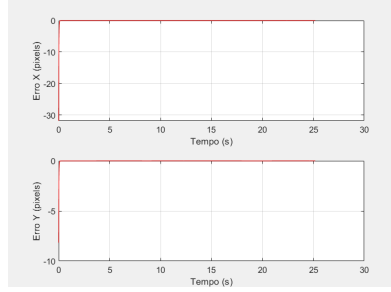
(a) Trajetórias nos eixos X e Y



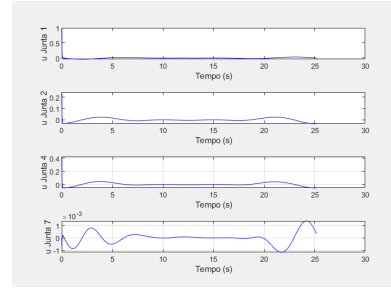
(b) Trajetória no plano X-Y



(c) Trajetória no espaço XYZ



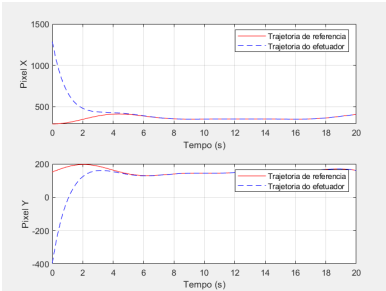
(d) Erro de posição nos eixos X e Y



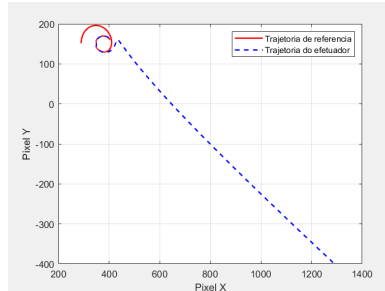
(e) Sinais de controle das juntas

Figura 9: Gráficos do controle por servo-visão ($K = 37.25$)

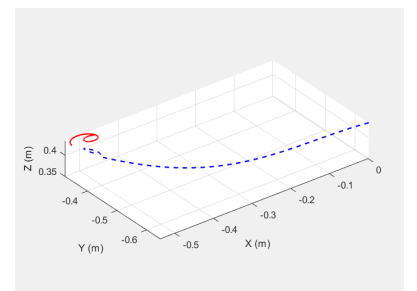
É possível notar que o erro vai muito mais rápido para zero, mas possui um sinal de controle das juntas muito maior no começo do seguimento da trajetória. Além disso, foi feita outra verificação de desempenho do controlador projetado se a câmera é rotacionada em $\phi = \pi/3$ mas esta rotação não é considerada na lei de controle u , i.e. em u tem-se $K_p(\phi = 0)$. Foi obtido o seguinte resultado:



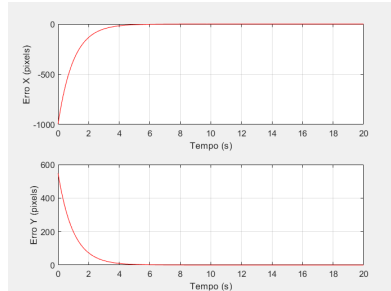
(a) Trajetórias nos eixos X e Y



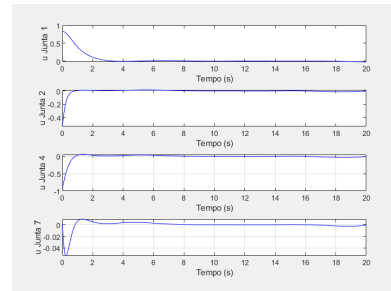
(b) Trajetória no plano X-Y



(c) Trajetória no espaço XYZ



(d) Erro de posição nos eixos X e Y



(e) Sinais de controle das juntas

Figura 10: Gráficos do controle por servo-visão ($K = 1$, $\phi = \pi/3$ e $K_p(\phi = 0)$)

Ao analisar os gráficos de trajetórias nos eixos X e Y ao longo tempo e do erro posição nos eixos X e Y dá uma falsa impressão de que o controle funcionou da forma correta. Porém, fica claro pelos outros gráficos que a trajetória do efetuador divergiu da trajetória da referência. Porque uma mudança na orientação da câmera sem alterar o controle acaba gerando uma total não conformidade com controle. Sendo que deveria ter o seguinte comportamento ao alterar a lei de controle para $\phi = \pi/3$.

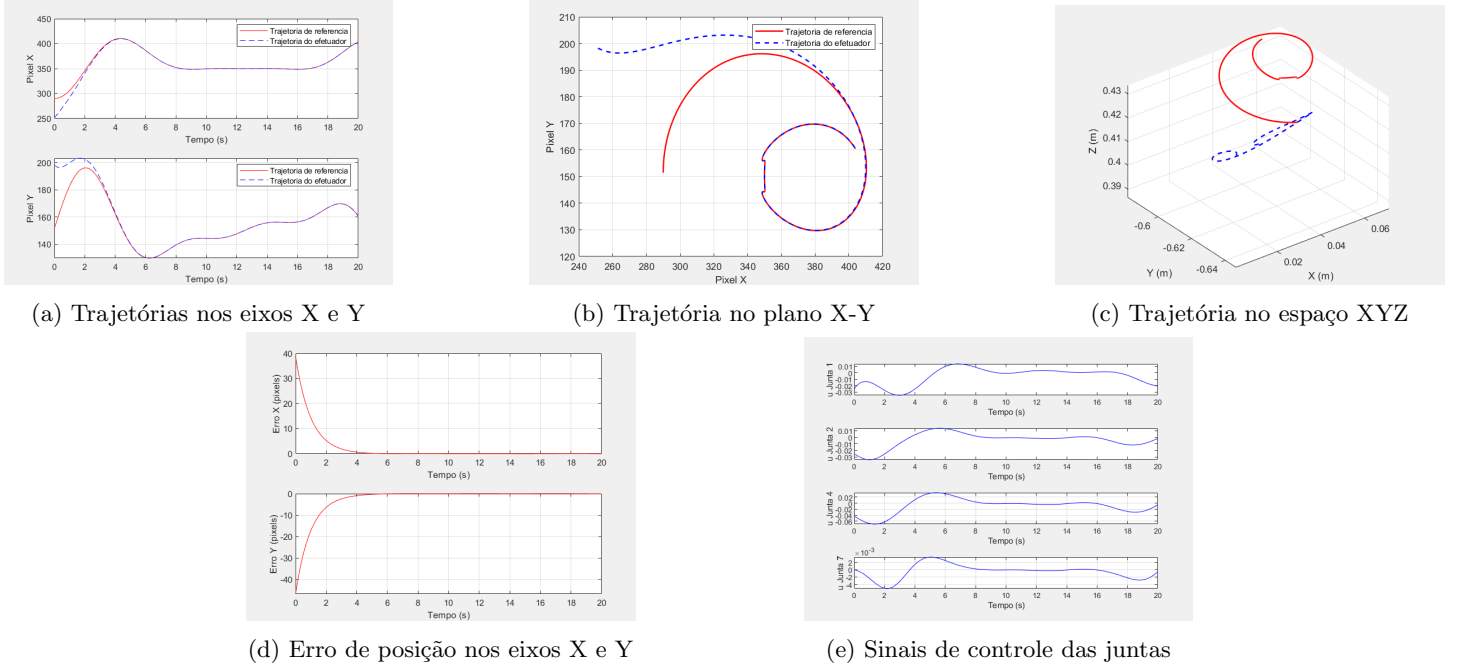


Figura 11: Gráficos do controle por servo-visão ($K = 1$, $\phi = \pi/3$ e $K_p(\phi = \pi/3)$)

3 Conclusão

Em resumo, os dois problemas abordados envolvem o controle cinemático de manipuladores com diferentes abordagens. No primeiro caso, foi possível simular o manipulador Kinova Gen3 e avaliar o desempenho dos controladores, destacando a importância da sintonia dos ganhos para garantir estabilidade e precisão nas trajetórias. No segundo caso, o controle por servo-visão foi ajustado para garantir que o manipulador seguisse a trajetória desejada, considerando as limitações da câmera e a rotação não considerada na lei de controle. Ambos os casos ressaltam a importância do planejamento e da sintonia dos controladores para um desempenho estável e preciso.

Apêndice

Listing 1: Código da primeira parte do trabalho

```

1 clear all;
2
3 % Definicao dos parametros DH com juntas revolutas
4 L(1) = Revolute('d', -0.2848, 'a', 0, 'alpha', pi/2, 'offset', 0);
5 L(2) = Revolute('d', -0.0118, 'a', 0, 'alpha', pi/2, 'offset', pi);
6 L(3) = Revolute('d', -0.4208, 'a', 0, 'alpha', pi/2, 'offset', pi);
7 L(4) = Revolute('d', -0.0128, 'a', 0, 'alpha', pi/2, 'offset', pi);
8 L(5) = Revolute('d', -0.3143, 'a', 0, 'alpha', pi/2, 'offset', pi);
9 L(6) = Revolute('d', 0, 'a', 0, 'alpha', pi/2, 'offset', pi);
10 L(7) = Revolute('d', 0, 'a', 0, 'alpha', pi, 'offset', pi);
11
12 % Criacao do manipulador
13 kinova = SerialLink(L, 'name', 'Kinova');
14 kinova.base = trotx(180); % Define a base do robo
15
16 % Condicoes iniciais
17 q0 = deg2rad([0, 15, 180, -130, 0, 55, 90]);
18
19 % Inicializacao de variaveis
20 qi = q0;
21 h = 0.01; % Passo de tempo
22 tmax = 20; % Tempo maximo (s)
23 wmax = 1; % Velocidade maxima das juntas (rad/s)
24 wn = pi/2; % Frequencia natural
25 wn3 = pi/8; % Frequencia natural - Terceira Trajetoria
26
27 K = 1.81; % Ganho do controlador 1 da trajetoria (a)
28 % K = 2.05; % Ganho do controlador 1 da trajetoria (b)
29 % K = 1.99; % Ganho do controlador 1 da trajetoria (c)
30 % K = 1.39; % Ganho do controlador 2 da trajetoria (a)
31 % K = 2.42; % Ganho do controlador 2 da trajetoria (b)
32 % K = 1.10; % Ganho do controlador 2 da trajetoria (c)
33
34 % Vetores para armazenamento dos dados
35 t_vec = 0:h:tmax;
36 x_ref = zeros(3, length(t_vec));
37 x_out = zeros(3, length(t_vec));
38 u_vec = zeros(4, length(t_vec)); % Para armazenar os 4 primeiros valores de u
39
40 % Simulacao
41 for k = 1:length(t_vec)
42     t = t_vec(k);

```

```

43
44 % Jacobiano e cinematica direta
45 J = kinova.jacob0(qi); % Jacobiano do robo
46 Jp = J(1:3,1:7);
47 x = kinova.fkine(qi).t; % Posicao atual do fim do braco
48
49 % Selecionar a trajetoria desejada:
50 xdd=0;
51
52 % Trajetoria (a)
53 xd = routeGen1(t, wn);
54 % xdd = routeGen1Deriv(t, wn);
55
56 % Trajetoria (b)
57 % xd = routeGen2(t, wn);
58 % xdd = routeGen2Deriv(t, wn);
59
60 % Trajetoria (c)
61 % xd = routeGen3(t, wn3);
62 % xdd = routeGen3Deriv(t, wn3);
63
64 % Erro de posicao e velocidade desejada
65 err = xd - x; % Erro de posicao
66
67 % Controlador
68 u = pinv(Jp) * (xdd + K * err); % Calculo das velocidades para as primeiras 4 juntas
69
70 % Verificar se algum valor de u e maior que 1
71 if any(abs(u) > 1)
72     disp('Alerta: Um ou mais valores de u excedem 1.');
```

73 disp('Valores de u:');

74 disp(u);

75 end

76

77 % Integracao (Euler) para as primeiras 4 juntas

78 qk = qi; % Copia o vetor atual de juntas

79 qk = qi + h * u'; % Atualiza apenas as primeiras 4 juntas

80 qi = qk; % Atualiza o estado

81

82 % Armazenar dados

83 x_ref(:, k) = xd; % Trajetoria de referencia

84 x_out(:, k) = x; % Trajetoria saida

85 u_vec(:, k) = u(1:4); % Armazena os 4 primeiros valores de u

86 end

87

88 % Plotagem dos resultados

```
89 figure;
90 subplot(3, 1, 1);
91 plot(t_vec, x_ref(1, :), 'r', 'LineWidth', 1.5); hold on;
92 plot(t_vec, x_out(1, :), 'b--', 'LineWidth', 1.5);
93 xlabel('Tempo (s)');
94 ylabel('Posicao X (m)');
95 legend('Referencia', 'Saida');
96 grid on;
97
98 subplot(3, 1, 2);
99 plot(t_vec, x_ref(2, :), 'r', 'LineWidth', 1.5); hold on;
100 plot(t_vec, x_out(2, :), 'b--', 'LineWidth', 1.5);
101 xlabel('Tempo (s)');
102 ylabel('Posicao Y (m)');
103 legend('Referencia', 'Saida');
104 grid on;
105
106 subplot(3, 1, 3);
107 plot(t_vec, x_ref(3, :), 'r', 'LineWidth', 1.5); hold on;
108 plot(t_vec, x_out(3, :), 'b--', 'LineWidth', 1.5);
109 xlabel('Tempo (s)');
110 ylabel('Posicao Z (m)');
111 legend('Referencia', 'Saida');
112 grid on;
113
114 % Grafico 3D da trajetoria real e desejada
115 figure;
116 plot3(x_ref(1, :), x_ref(2, :), x_ref(3, :), 'r-', 'LineWidth', 1.5); hold on;
117 plot3(x_out(1, :), x_out(2, :), x_out(3, :), 'b--', 'LineWidth', 1.5);
118 xlabel('X (m)');
119 ylabel('Y (m)');
120 zlabel('Z (m)');
121 legend('Referencia', 'Saida');
122 grid on;
123
124 % Graficos das velocidades das 4 primeiras juntas
125 figure;
126
127 subplot(4, 1, 1);
128 plot(t_vec, u_vec(1, :), 'r', 'LineWidth', 1.5);
129 xlabel('Tempo (s)');
130 ylabel('u_1 (rad/s)');
131 grid on;
132
133 subplot(4, 1, 2);
134 plot(t_vec, u_vec(2, :), 'b', 'LineWidth', 1.5);
```

```
135 xlabel('Tempo (s)');
136 ylabel('u_2 (rad/s)');
137 grid on;
138
139 subplot(4, 1, 3);
140 plot(t_vec, u_vec(3, :), 'g', 'LineWidth', 1.5);
141 xlabel('Tempo (s)');
142 ylabel('u_3 (rad/s)');
143 grid on;
144
145 subplot(4, 1, 4);
146 plot(t_vec, u_vec(4, :), 'm', 'LineWidth', 1.5);
147 xlabel('Tempo (s)');
148 ylabel('u_4 (rad/s)');
149 grid on;
150
151 % Calculo do erro ao longo do tempo
152 error_x = x_ref(1, :) - x_out(1, :); % Erro no eixo X
153 error_y = x_ref(2, :) - x_out(2, :); % Erro no eixo Y
154 error_z = x_ref(3, :) - x_out(3, :); % Erro no eixo Z
155
156 % Graficos do erro em subplots
157 figure;
158
159 subplot(3, 1, 1);
160 plot(t_vec, error_x, 'r', 'LineWidth', 1.5);
161 xlabel('Tempo (s)');
162 ylabel('Erro X (m)');
163 grid on;
164
165 subplot(3, 1, 2);
166 plot(t_vec, error_y, 'g', 'LineWidth', 1.5);
167 xlabel('Tempo (s)');
168 ylabel('Erro Y (m)');
169 grid on;
170
171 subplot(3, 1, 3);
172 plot(t_vec, error_z, 'b', 'LineWidth', 1.5);
173 xlabel('Tempo (s)');
174 ylabel('Erro Z (m)');
175 grid on;
176
177 % Funcoes de geracao de trajetoria
178 function xd = routeGen1(t, wn)
179     % Trajetoria (a): Circunferencia no plano X-Z
180     r = 0.1; % Raio da circunferencia
```

```
181     x0 = [0.4; 0; 0.4]; % Centro da circunferencia
182     xd = x0 + [r * cos(wn * t); 0; r * sin(wn * t)];
183 end
184
185 function xdot = routeGen1Deriv(t, wn)
186     % Derivada da trajetoria (a): Velocidade da circunferencia no plano X-Z
187     r = 0.1; % Raio da circunferencia
188     xdot = [-r * wn * sin(wn * t); 0; r * wn * cos(wn * t)];
189 end
190
191 function xd = routeGen2(t, wn)
192     % Trajetoria (b): Espiral no plano X-Z
193     r = 0.1; % Raio inicial
194     x0 = [0.4; 0; 0.4]; % Centro da espiral
195     xd = x0 + [r * cos(wn * t); 0; r * sin(wn * t)];
196 end
197
198 function xdot = routeGen2Deriv(t, wn)
199     % Derivada da trajetoria (b): Velocidade da espiral no plano X-Z
200     r = 0.1; % Raio inicial
201     xdot = [-r * wn * sin(wn * t); 0; r * wn * cos(wn * t)];
202 end
203
204 function xd = routeGen3(t, wn)
205     % Trajetoria (c): Linha reta no eixo X
206     x0 = [0.4; 0; 0.4]; % Ponto inicial
207     xf = [0.5; 0; 0.5]; % Ponto final
208     xd = x0 + (xf - x0) * sin(wn * t);
209 end
210
211 function xdot = routeGen3Deriv(t, wn)
212     % Derivada da trajetoria (c): Velocidade da linha reta no eixo X
213     x0 = [0.4; 0; 0.4]; % Ponto inicial
214     xf = [0.5; 0; 0.5]; % Ponto final
215     xdot = (xf - x0) * cos(wn * t) * wn;
216 end
```

Listing 2: Código da segunda parte do trabalho

```

1 clear all; close all;
2
3 % Definicao dos parametros DH com juntas revolutas
4 L(1) = Revolute('d', -0.2848, 'a', 0, 'alpha', pi/2, 'offset', 0); % Parametro da junta 1
5 L(2) = Revolute('d', -0.0118, 'a', 0, 'alpha', pi/2, 'offset', pi); % Parametro da junta 2
6 L(3) = Revolute('d', -0.4208, 'a', 0, 'alpha', pi/2, 'offset', pi); % Parametro da junta 3
7 L(4) = Revolute('d', -0.0128, 'a', 0, 'alpha', pi/2, 'offset', pi); % Parametro da junta 4
8 L(5) = Revolute('d', -0.3143, 'a', 0, 'alpha', pi/2, 'offset', pi); % Parametro da junta 5
9 L(6) = Revolute('d', 0, 'a', 0, 'alpha', pi/2, 'offset', pi); % Parametro da junta 6
10 L(7) = Revolute('d', -0.3574, 'a', 0, 'alpha', pi, 'offset', pi); % Parametro da junta 7
11
12 % Criacao do manipulador
13 kinova_gen3 = SerialLink(L, 'name', 'Kinova'); % Criacao do modelo do manipulador
14 kinova_gen3.base = trotx(180); % Define a base do robo (rotacao em torno do eixo X)
15
16 % Parametros do sistema
17 f = 4e-3; % Distancia focal da camera (m)
18 z0 = 0.4338; % Profundidade inicial (m)
19 alpha = 200000; % Fator de escala (pixels/m)
20 Oc = [320; 240]; % Origem do plano da imagem (pixels)
21 Pbc = [0; -0.603; 0]; % Posicao da camera em relacao a base (m)
22 phi = 0; % Angulo de orientacao da camera (rad)
23 phi_trocado = pi/3; % Angulo de orientacao da camera trocado (rad)
24 wn = 0.5; % Frequencia angular (rad/s)
25
26 % Ganho do controlador
27 K = 1; % Ajuste o ganho conforme necessario para o controlador
28
29 % Trajetoria desejada no plano da imagem
30 h = 0.01; % Passo de tempo
31 tmax = 20; % Tempo maximo (s)
32 t = 0:h:tmax; % Vetor de tempo para simulacao
33 xcd = [
34     360- 30*cos(wn*t)- 30*cos(1.5*wn*t)- 10*cos(2*wn*t); % Trajetoria no eixo X
35     150- 20*cos(wn*t + 1.6)- 20*cos(1.5*wn*t + 1.6)- 10*cos(2*wn*t + 1.6) % Trajetoria no eixo Y
36 ];
37 xcdd = [
38     30*sin(wn*t)*wn + 30*sin(1.5*wn*t)*1.5*wn + 10*sin(2*wn*t)*2*wn; % Derivada da trajetoria no eixo X
39     20*sin(wn*t + 1.6)*wn + 20*sin(1.5*wn*t + 1.6)*1.5*wn + 10*sin(2*wn*t + 1.6)*2*wn % Derivada da
    trajetoria no eixo Y
40 ];
41
42 % Transformacao camera/espaco de trabalho
43 Kp = (f / z0) * diag([alpha,-alpha]) * [cos(phi), sin(phi); sin(phi),-cos(phi)];

```

```

44 % Kp = (f / z0) * diag([alpha,-alpha]) * [cos(phi_trocado), sin(phi_trocado); sin(phi_trocado),-cos(
    phi_trocado)];
45 % Matriz de projecao da camera no plano da imagem
46 xc0 = (f / z0) * diag([alpha,-alpha]) * [-cos(phi),-sin(phi);-sin(phi), cos(phi)]*Pbc(1:2) + 0c;
47 % xc0 = (f / z0) * diag([alpha,-alpha]) * [-cos(phi_trocado),-sin(phi_trocado);-sin(phi_trocado), cos(
    phi_trocado)]*Pbc(1:2) + 0c;
48 % PosiÃ§Ã£o inicial da camera no plano da imagem
49
50 % Inicializacao de variaveis
51 xc = zeros(2, length(t)-1); % Posicao no plano da imagem
52 theta = deg2rad([90; 15; 180;-130; 0; 55; 0]); % Angulos iniciais das juntas (rad)
53 theta_log = zeros(length(theta), length(t)-1); % Log para salvar os valores de theta
54 xyz = zeros(3, length(t)-1); % Posicao no espaco do efetuador (x, y, z)
55
56 % Erros
57 ex = zeros(1, length(t)-1); % Erro no eixo X
58 ey = zeros(1, length(t)-1); % Erro no eixo Y
59 u_log = zeros(4, length(t)-1); % Log para salvar os valores de u
60
61 % Simulacao do movimento
62 for k = 1:length(t)-1
63     % Cinematica direta (posicao no espaco de trabalho)
64     x = kinova_gen3.fkine(theta).t; % Calcula a posicao do efetuador
65     xyz(:, k) = x; % Salva a posicao do efetuador no espaco (x, y, z)
66
67     % Projecao no plano da imagem
68     xc(:, k) = Kp * x(1:2) + xc0; % Calcula a posicao projetada no plano da imagem
69
70     % Erro na imagem (diferenca entre a trajetoria desejada e a atual)
71     ecd = xcd(:, k) - xc(:, k);
72     ex(k) = ecd(1); % Erro no eixo X
73     ey(k) = ecd(2); % Erro no eixo Y
74
75     % Jacobiano do manipulador (somente juntas controladas)
76     J = kinova_gen3.jacob0(theta); % Jacobiano completo
77     J_reduced = J(1:2, [1, 2, 4, 6]); % Considera apenas as colunas das juntas 1, 2, 4 e 6
78
79     % Controle cinemático
80     Jv = Kp * J_reduced; % Controle baseado no Jacobiano reduzido
81     xcd_dot = xcd(:, k); % Derivada da trajetoria desejada
82     u = pinv(Jv) * (xcd_dot + K * ecd); % Calcula o comando de controle
83
84     % Verificar se algum valor de u é maior que 1
85     if any(abs(u) > 1)
86         disp('Alerta: Um ou mais valores de u excedem 1.');
```

```

88         disp(u);
89     end
90
91     % Atualizacao dos angulos das juntas
92     theta_log(:, k) = theta; % Salva os angulos das juntas
93     theta([1, 2, 4, 6]) = theta([1, 2, 4, 6]) + u * h; % Atualiza os angulos com o controle
94
95     % Log de u
96     u_log(:, k) = u;
97 end
98
99 % Graficos para visualizacao dos resultados
100 figure;
101 subplot(2, 1, 1);
102 plot(t(1:end-1), xcd(1, 1:end-1), 'r', t(1:end-1), xc(1, :), 'b--');
103 xlabel('Tempo (s)'); ylabel('Pixel X');
104 legend('Trajetoria de referencia', 'Trajetoria do efetuador');
105 grid on;
106
107 subplot(2, 1, 2);
108 plot(t(1:end-1), xcd(2, 1:end-1), 'r', t(1:end-1), xc(2, :), 'b--');
109 xlabel('Tempo (s)'); ylabel('Pixel Y');
110 legend('Trajetoria de referencia', 'Trajetoria do efetuador');
111 grid on;
112
113 figure; % Grafico XY (trajetoria no plano da tela)
114 plot(xcd(1, :), xcd(2, :), 'r', 'LineWidth', 1.5); hold on;
115 plot(xc(1, :), xc(2, :), 'b--', 'LineWidth', 1.5);
116 xlabel('Pixel X'); ylabel('Pixel Y');
117 legend('Trajetoria de referencia', 'Trajetoria do efetuador');
118 grid on;
119
120 % Grafico XYZ (trajetoria no espaco do efetuador)
121 xdr = pinv(Kp)*(xcd-xc0);
122 zdr = z0*ones(size(xcd(1,:)));
123 figure;
124 plot3(xdr(1, :), xdr(2, :), zdr, 'r-', 'LineWidth', 1.5); hold on;
125 plot3(xyz(1, :), xyz(2, :), xyz(3, :), 'b--', 'LineWidth', 1.5);
126 xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');
127 grid on; axis equal;
128
129 % Grafico de erro
130 figure;
131 subplot(2, 1, 1);
132 plot(t(1:end-1), ex, 'r', 'LineWidth', 1.5);
133 xlabel('Tempo (s)'); ylabel('Erro no X');

```



```
134 grid on;
135
136 subplot(2, 1, 2);
137 plot(t(1:end-1), ey, 'r', 'LineWidth', 1.5);
138 xlabel('Tempo (s)'); ylabel('Erro no Y');
139 grid on;
```